

Guardian Escolar - Funcionamiento de la app movil MAUI

Este documento explica el funcionamiento actual de la aplicacion movil de **Guardian Escolar**, desarrollada con **.NET MAUI**. La idea es que cualquier persona que lo lea pueda entender que hace la app, como esta organizada, que funciones tiene, para que sirve cada parte y cuales son los codigos importantes.

Ruta del proyecto movil:

```
dvlp-front/dvlp-movil/app-movil
```

Archivo principal del proyecto:

```
dvlp-front/dvlp-movil/app-movil/app-movil.csproj
```

1. Objetivo de la aplicacion movil

La app movil de Guardian Escolar busca apoyar el seguimiento del transporte escolar desde el celular. Su objetivo principal es permitir que usuarios como acudientes, estudiantes o personal autorizado puedan consultar informacion relacionada con rutas, ubicacion, conductor, vehiculo y estado del recorrido.

En el estado actual, la aplicacion ya cuenta con:

- Flujo visual de inicio de sesion.
- Flujo visual para recuperar contrasena.
- Pantalla principal con mapa.
- Buscador de ruta.
- Boton visual de notificaciones.
- Tarjeta con informacion de una ruta.
- Barra inferior de navegacion.
- Pantalla base de perfil.
- Soporte de traducciones por archivos **.resx**.
- Componentes reutilizables para botones, entradas, codigo de verificacion, busqueda, tarjeta de ruta y navegacion inferior.

Importante: varias funciones ya estan construidas a nivel visual y de navegacion, pero todavia no estan conectadas completamente con el backend ni con datos reales.

2. Tecnologia usada

La aplicacion esta hecha con **.NET MAUI**, que permite crear aplicaciones moviles usando C# y XAML.

Configuracion principal del archivo **app-movil.csproj**:

```
<TargetFrameworks>net10.0-android</TargetFrameworks>
<OutputType>Exe</OutputType>
<RootNamespace>app_movil</RootNamespace>
<UseMaui>true</UseMaui>
<SingleProject>true</SingleProject>
<ImplicitUsings>enable</ImplicitUsings>
<Nullable>enable</Nullable>
<MauiXamlInflator>SourceGen</MauiXamlInflator>
```

Esto significa:

- Actualmente la app compila para Android.
- Usa el modelo de proyecto unico de MAUI.
- El namespace principal es `app_movil`.
- Las vistas estan construidas con XAML.
- La nulabilidad esta activada para mejorar la seguridad del codigo.
- La compilacion de XAML usa `SourceGen`.

3. Dependencias principales

El proyecto usa estas librerias:

```
<PackageReference Include="Mapsui.Maui" Version="5.1.0" />
<PackageReference Include="Microsoft.Maui.Controls" Version="$(MauiVersion)" />
<PackageReference Include="Microsoft.Extensions.Logging.Debug" Version="10.0.0" />
<PackageReference Include="UraniumUI.Icons.MaterialSymbols" Version="2.15.0" />
<PackageReference Include="UraniumUI.Material" Version="2.15.0" />
```

Funcion de cada dependencia:

- `Microsoft.Maui.Controls`: base de controles visuales de MAUI.
- `Microsoft.Extensions.Logging.Debug`: permite ver logs durante el desarrollo.
- `UraniumUI.Material`: aporta estilos y componentes visuales tipo Material.
- `UraniumUI.Icons.MaterialSymbols`: permite usar iconos Material Symbols.
- `Mapsui.Maui`: permite mostrar mapas dentro de la aplicacion.

4. Inicio de la aplicacion

El archivo de arranque es:

```
MauiProgram.cs
```

Alli se configura MAUI, UraniumUI, SkiaSharp y las fuentes:

```
builder
    .UseMauiApp<App>()
    .UseUraniumUI()
    .UseUraniumUIMaterial()
    .UseSkiaSharp()
    .ConfigureFonts(fonts =>
    {
        fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
        fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
        fonts.AddMaterialSymbolsFonts();
    });
```

Esto permite:

- Cargar la clase principal **App**.
- Habilitar estilos de UraniumUI.
- Habilitar controles graficos necesarios para Mapsui mediante SkiaSharp.
- Registrar las fuentes OpenSans.
- Registrar iconos Material Symbols.
- Activar logs de depuracion cuando se compila en modo **DEBUG**.

5. Clase App y recursos globales

Archivos:

```
App.xaml
App.xaml.cs
```

App.xaml carga los recursos globales:

```
<ResourceDictionary Source="Resources/Styles/Colors.xaml" />
<ResourceDictionary Source="Resources/Styles/Styles.xaml" />
```

App.xaml.cs fuerza el tema claro y crea la ventana principal:

```
UserAppTheme = AppTheme.Light;
return new Window(new AppShell());
```

Funcionamiento:

- La app inicia con tema claro.
- La navegacion principal se maneja desde **AppShell**.
- Los colores, estilos, fuentes y recursos quedan disponibles para todas las pantallas.

6. Navegacion con AppShell

Archivos:

```
AppShell.xaml
AppShell.xaml.cs
```

`AppShell.xaml` define las pantallas principales visibles para Shell:

```
<ShellContent
    Title="Welcome"
    ContentTemplate="{DataTemplate auth:Welcome}"
    Route="Welcome" />

<ShellContent
    Title="MainPage"
    ContentTemplate="{DataTemplate home:MainPage}"
    Route="MainPage" />
```

Rutas registradas en `AppShell.xaml.cs`:

```
Routing.RegisterRoute(nameof(ForgotPassword), typeof(ForgotPassword));
Routing.RegisterRoute(nameof(VerifyCode), typeof(VerifyCode));
Routing.RegisterRoute(nameof(NewPassword), typeof(NewPassword));
Routing.RegisterRoute(nameof(Profile), typeof(Profile));
```

Pantallas dentro de la navegacion:

- `Welcome`: pantalla inicial de inicio de sesion.
- `MainPage`: pantalla principal con mapa.
- `ForgotPassword`: pantalla para solicitar recuperacion de contrasena.
- `VerifyCode`: pantalla para ingresar codigo de verificacion.
- `NewPassword`: pantalla para crear nueva contrasena.
- `Profile`: pantalla base de perfil.

7. Flujo general de usuario

El flujo actual de la app es:

1. El usuario abre la app.
2. La app carga `AppShell`.
3. La primera pantalla visible es `Welcome`.
4. Desde `Welcome`, el usuario puede:
 - Presionar `Ingresar` para ir a `MainPage`.
 - Presionar `Olvido la contrasena` para ir a `ForgotPassword`.

5. Desde `ForgotPassword`, el usuario ingresa su correo y pasa a `VerifyCode`.
6. Desde `VerifyCode`, el usuario ingresa el código y pasa a `NewPassword`.
7. Desde `NewPassword`, el usuario restaura la contraseña y vuelve visualmente a `Welcome`.
8. En `MainPage`, el usuario ve el mapa, buscador, botón de notificaciones, información de ruta y barra inferior.

Este flujo todavía es principalmente visual. No hay validación real de credenciales, correo, código o contraseña contra el backend.

8. Módulo de autenticación

Ruta:

```
Features/Auth
```

Pantallas:

```
Features/Auth/Views/Welcome.xaml  
Features/Auth/Views/ForgotPassword.xaml  
Features/Auth/Views/VerifyCode.xaml  
Features/Auth/Views/NewPassword.xaml
```

Code-behind:

```
Features/Auth/ViewModels/Welcome.xaml.cs  
Features/Auth/ViewModels/ForgotPassword.xaml.cs  
Features/Auth/ViewModels/VerifyCode.xaml.cs  
Features/Auth/ViewModels/NewPassword.xaml.cs
```

Nota importante: aunque la carpeta se llama `ViewModels`, esos archivos todavía son code-behind, no `ViewModels` reales. Funcionan porque mantienen el namespace de las vistas y heredan de `ContentPage`.

8.1. Welcome

Archivos:

```
Features/Auth/Views/Welcome.xaml  
Features/Auth/ViewModels/Welcome.xaml.cs
```

Función:

- Muestra la pantalla de inicio de sesión.
- Tiene campo de correo.
- Tiene campo de contraseña.

- Tiene enlace para recuperar contraseña.
- Tiene boton para ingresar.

Componentes usados:

- `InputField` para correo.
- `InputField` para contraseña.
- `PrimaryButton` para ingresar.
- Traducciones con `{services:Translation ...}`.

Navegacion:

```
await Shell.Current.GoToAsync(nameof(ForgotPassword));  
await Shell.Current.GoToAsync("//MainPage");
```

Estado actual:

- La pantalla esta construida.
- El boton `Ingresar` lleva a la pantalla principal.
- No valida usuario ni contraseña contra backend.
- No guarda sesion ni token.

8.2. ForgotPassword

Archivos:

```
Features/Auth/Views/ForgotPassword.xaml  
Features/Auth/ViewModels/ForgotPassword.xaml.cs
```

Funcion:

- Permite ingresar un correo para recuperar contraseña.
- Muestra instrucciones al usuario.
- Tiene boton para enviar codigo.

Navegacion:

```
await Shell.Current.GoToAsync(nameof(VerifyCode));
```

Estado actual:

- La pantalla esta construida.
- El boton lleva a la pantalla de verificacion.
- No envia un correo real.
- No consume endpoint del backend.

8.3. VerifyCode

Archivos:

```
Features/Auth/Views/VerifyCode.xaml  
Features/Auth/ViewModels/VerifyCode.xaml.cs
```

Funcion:

- Permite ingresar un codigo de verificacion.
- Usa el componente `CodeInput`.
- Tiene texto para reenviar codigo.
- Tiene boton para continuar.

Navegacion:

```
await Shell.Current.GoToAsync(nameof(NewPassword));
```

Estado actual:

- La pantalla esta construida.
- El flujo continua hacia nueva contrasena.
- El codigo no se valida contra backend.
- El texto de reenviar codigo es visual; no tiene temporizador funcional.

8.4. NewPassword

Archivos:

```
Features/Auth/Views/NewPassword.xaml  
Features/Auth/ViewModels/NewPassword.xaml.cs
```

Funcion:

- Permite ingresar nueva contrasena.
- Permite confirmar contrasena.
- Tiene boton para restaurar.

Estado actual:

- La pantalla esta construida.
- Usa campos de contrasena.
- No valida que las contrasenas coincidan.
- No actualiza la contrasena en backend.
- Actualmente vuelve a `Welcome` usando `Navigation.PushAsync(new Welcome());` se recomienda unificarlo con `Shell`.

9. Pantalla principal con mapa

Ruta:

```
Features/Home/Views/MainPage.xaml
Features/Home/Views/MainPage.xaml.cs
```

Funcion:

- Muestra un mapa principal.
- Carga capa de OpenStreetMap.
- Centra la vista aproximada sobre Colombia.
- Muestra una barra superior con buscador y notificaciones.
- Muestra una tarjeta inferior con informacion de ruta.
- Muestra una barra inferior con tres opciones: rutas, ubicacion y perfil.

Codigo principal del mapa:

```
var map = new Mapsui.Map();
map.Widgets.Clear();
map.Layers.Add(OpenStreetMap.CreateTileLayer());
RouteMap.Map = map;
```

Ubicacion inicial:

```
var (minX, minY) = SphericalMercator.FromLonLat(-81.85, -4.23);
var (maxX, maxY) = SphericalMercator.FromLonLat(-66.85, 13.51);
var bbox = new MRect(minX, minY, maxX, maxY);
map.Navigator.ZoomToBox(bbox);
```

Estado actual:

- El mapa esta implementado con Mapsui.
- La capa visual viene de OpenStreetMap.
- El mapa necesita internet para cargar correctamente los tiles.
- Todavia no muestra rutas reales, buses reales ni marcadores.
- Todavia no consume ubicacion GPS del dispositivo.
- La barra inferior esta en pantalla, pero el evento `TabSelected` no aparece enlazado en el XAML de `MainPage`.

10. Perfil

Archivos:


```
Features/Profile/Views/Profile.xaml  
Features/Profile/Views/Profile.xaml.cs
```

Estado actual:

- Existe una pantalla base de perfil.
- La pantalla muestra un texto de prueba.
- La ruta `Profile` esta registrada en `AppShell.xaml.cs`.
- El namespace actual es `app_movil.Features.Home.Views`, aunque el archivo esta dentro de `Features/Profile/Views`.

Recomendacion:

- Ajustar el namespace a `app_movil.Features.Profile.Views`.
- Agregar la pantalla como `ShellContent` si se quiere navegar con ruta absoluta.
- Conectar la barra inferior para abrir el perfil.
- Reemplazar el contenido de prueba por datos reales del usuario.

11. Componentes reutilizables

Los componentes estan en:

```
Components
```

11.1. PrimaryButton

Archivos:

```
Components/Buttons/PrimaryButton.xaml  
Components/Buttons/PrimaryButton.xaml.cs
```

Funcion:

- Boton reutilizable para acciones principales.
- Permite configurar texto.
- Permite usar `Command`.
- Permite usar `CommandParameter`.
- Expone evento `Clicked`.

Se usa en:

- `Welcome`
- `ForgotPassword`
- `VerifyCode`
- `NewPassword`

11.2. NotificationButton

Archivos:

```
Components/Buttons/NotificationButton.xaml  
Components/Buttons/NotificationButton.xaml.cs
```

Funcion:

- Boton visual de notificaciones.
- Muestra un icono de campana.
- Muestra un punto rojo como indicador de alerta o notificacion pendiente.

Estado actual:

- Es visual.
- No tiene evento publico de click.
- No abre una pantalla de notificaciones.
- No consume alertas reales.

11.3. InputField

Archivos:

```
Components/Inputs/InputField.xaml  
Components/Inputs/InputField.xaml.cs
```

Funcion:

- Campo reutilizable para entrada de texto.
- Permite label superior.
- Permite placeholder.
- Permite enlace bidireccional de texto.
- Permite definir teclado.
- Permite marcar campo como contrasena.

Propiedades principales:

```
LabelText  
Placeholder  
Text  
Keyboard  
IsPassword
```

11.4. CodeInput

Archivos:

```
Components/Inputs/CodeInput.xaml  
Components/Inputs/CodeInput.xaml.cs
```

Funcion:

- Componente para ingresar codigo de verificacion.
- Muestra seis campos.
- Cada campo acepta un caracter.
- Usa teclado numerico.

Estado actual:

- Sirve como componente visual.
- Todavia no expone una propiedad unica con el codigo completo.
- Todavia no mueve automaticamente el foco entre casillas.

11.5. SearchInput

Archivos:

```
Components/Inputs/SearchInput.xaml  
Components/Inputs/SearchInput.xaml.cs
```

Funcion:

- Buscador visual para rutas.
- Muestra icono de busqueda.
- Usa placeholder traducido con la clave `SearchRoute`.

Estado actual:

- Es visual.
- No expone todavia una propiedad `Text`.
- No filtra rutas.
- No consulta rutas desde backend.

11.6. BottomTabBar

Archivos:

```
Components/Layout/BottomTabBar.xaml  
Components/Layout/BottomTabBar.xaml.cs
```

Funcion:

- Barra inferior de navegacion.
- Tiene tres opciones:
 - Rutas.
 - Ubicacion.
 - Perfil.

Evento expuesto:

```
public event EventHandler<string> TabSelected;
```

Valores que emite:

```
routes  
location  
profile
```

Estado actual:

- El componente existe y emite eventos.
- En `MainPage.xaml.cs` existe el metodo `OnTabSelected`.
- En `MainPage.xaml` no se ve enlazado el evento `TabSelected`, por lo que al tocar las opciones puede no ejecutarse la navegacion.
- Las rutas absolutas `//routes` y `//location` no aparecen registradas actualmente.

11.7. RouteInfoCard

Archivos:

```
Components/Cards/RouteInfoCard.xaml  
Components/Cards/RouteInfoCard.xaml.cs
```

Funcion:

- Muestra informacion resumida de una ruta.
- Presenta nombre de ruta.
- Presenta conductor.
- Presenta placa del vehiculo.
- Presenta horario.
- Presenta cantidad de paradas.
- Presenta destino final.

Datos actuales de ejemplo:

```
Ruta Centro  
Carlos Perez  
ABC-123  
06:00 AM - 07:30 AM  
12 paradas  
Destino final: Escuela
```

Estado actual:

- Es una tarjeta visual.
- Los datos estan escritos directamente en el XAML.
- Todavia no recibe datos dinamicos desde backend.

12. Internacionalizacion y traducciones

Archivos:

```
Resources/Strings/AppStrings.es.resx  
Resources/Strings/AppStrings.en.resx  
Resources/Strings/AppStrings.fr.resx  
Resources/Strings/AppStrings.pt.resx
```

Servicio:

```
Core/Services/LocalizationService.cs
```

Extension XAML:

```
Core/Extensions/TranslationExtension.cs
```

Uso en XAML:

```
Text="{services:Translation Key=Login}"
```

Idiomas disponibles:

- Espanol.
- Ingles.
- Frances.
- Portugues.

Claves importantes:

```
Welcome
Email
Password
Login
ForgotPassword
ResetPassword
SendCode
Enter
EnterEmail
CodeVerif
CodeEmail
CodeSec
TransferCode
VerifyCode
NewPasswordDescription
NewPasswordTitle
ConfirmPasswordTitle
Restore
FinalDestination
Route
Location
Profile
School
SearchRoute
```

Observacion importante:

- Algunos textos aparecen con problemas de codificacion al leerlos desde consola, por ejemplo caracteres como Ã³ o Ã±.
- Se recomienda guardar XAML y `.resx` en UTF-8 correcto para que los acentos se vean bien en la app y en la documentacion.

13. Recursos visuales

13.1. Colores

Archivo:

```
Resources/Styles/Colors.xaml
```

Define colores globales como:

```
Primary
PrimaryDark
PrimaryLight
Background
CardBackground
CardSecondaryBackground
```

```
TextPrimary  
TextSecondary  
TitleColor  
InputPlaceholder  
BorderColor  
ButtonPrimary  
White  
Black  
Gray100...Gray950
```

13.2. Estilos

Archivo:

```
Resources/Styles/Styles.xaml
```

Define estilos globales para controles MAUI como:

```
Button  
Entry  
Label  
Border  
Page  
Shell  
NavigationPage  
TabbedPage  
SearchBar  
Switch  
Slider  
Picker  
DatePicker
```

Tambien define el estilo:

```
<Style x:Key="CodeEntryStyle" TargetType="Entry">
```

Ese estilo se usa en el componente `CodeInput`.

13.3. Fuentes

Archivos:

```
Resources/Fonts/OpenSans-Regular.ttf  
Resources/Fonts/OpenSans-Semibold.ttf
```

Alias registrados:

```
OpenSansRegular  
OpenSansSemibold
```

13.4. Icono y splash

Archivos:

```
Resources/AppIcon/appicon.svg  
Resources/AppIcon/appiconfg.svg  
Resources/Splash/splash.svg
```

Declarados en el `.csproj` como:

```
<MauiIcon Include="Resources\AppIcon\appicon.svg"  
ForegroundFile="Resources\AppIcon\appiconfg.svg" Color="#512BD4" />  
<MauiSplashScreen Include="Resources\Splash\splash.svg" Color="#512BD4"  
BaseSize="128,128" />
```

14. Plataformas soportadas

El proyecto contiene carpetas para varias plataformas:

```
Platforms/Android  
Platforms/iOS  
Platforms/MacCatalyst  
Platforms/Windows
```

Pero actualmente el `.csproj` solo compila para:

```
net10.0-android
```

Android

Archivo:

```
Platforms/Android/AndroidManifest.xml
```

Permisos:


```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
```

Estos permisos son importantes porque:

- La app necesita saber si hay conexion.
- El mapa de OpenStreetMap requiere internet para cargar.
- En el futuro, la app necesitara comunicarse con el backend.

15. Estado real de las funciones

Funcion	Estado actual	Observacion
Inicio de sesion	Visual y navegable	No valida credenciales reales
Recuperar contraseña	Visual y navegable	No envia correo real
Verificar codigo	Visual y navegable	No valida codigo real
Crear nueva contraseña	Visual	No actualiza backend
Mapa	Implementado visualmente	Usa OpenStreetMap, sin rutas reales
Busqueda de ruta	Visual	No filtra ni consulta datos
Notificaciones	Visual	No abre pantalla ni consulta alertas
Tarjeta de ruta	Visual	Datos estaticos
Barra inferior	Componente creado	Falta enlazar evento en <code>MainPage.xaml</code>
Perfil	Pantalla base	Contenido de prueba
Traducciones	Implementadas	Revisar codificacion UTF-8

16. Que funciona actualmente

Actualmente la app tiene funcionando:

- Estructura base de proyecto MAUI.
- Inicio desde `App`.
- Navegacion principal con `AppShell`.
- Pantalla inicial `Welcome`.
- Navegacion desde login hacia `MainPage`.
- Flujo visual de recuperacion de contraseña.
- Pantalla principal con mapa Mapsui.
- Capa de mapa OpenStreetMap.
- Buscador visual de rutas.
- Boton visual de notificaciones.
- Tarjeta visual de informacion de ruta.
- Barra inferior visual.

- Pantalla base de perfil.
- Componentes reutilizables.
- Recursos globales de estilos y colores.
- Traducciones por `.resx`.
- Configuración Android con permiso de internet.

17. Pendientes técnicos

Estos puntos son importantes para mejorar la app en una versión más completa:

1. Conectar autenticación con backend.
2. Guardar token o sesión del usuario.
3. Validar campos obligatorios en login y recuperación.
4. Validar formato de correo.
5. Validar que las contraseñas coincidan.
6. Enviar código de recuperación real.
7. Validar código de recuperación real.
8. Conectar mapa con rutas reales.
9. Mostrar marcadores de buses, paradas o estudiantes.
10. Consumir ubicación GPS cuando sea necesario.
11. Hacer que `RouteInfoCard` reciba datos dinámicos.
12. Hacer que `SearchInput` filtre o consulte rutas.
13. Conectar `NotificationButton` con alertas reales.
14. Enlazar `BottomAppBar.TabSelected` en `MainPage.xaml`.
15. Registrar correctamente las rutas de ubicación y perfil.
16. Corregir namespace de `Profile`.
17. Convertir code-behind en ViewModels reales si se quiere aplicar MVVM.
18. Corregir textos con codificación incorrecta.
19. Reducir advertencias de nulabilidad.
20. Agregar pruebas o validaciones manuales documentadas.

18. Riesgos o detalles a revisar

Navegación mezclada

La mayoría del flujo usa:

```
Shell.Current.GoToAsync(...)
```

Pero `NewPassword` usa:

```
Navigation.PushAsync(new Welcome());
```

Recomendación:

```
await Shell.Current.GoToAsync("//Welcome");
```

Asi se mantiene una sola forma de navegar.

Barra inferior no conectada

`BottomAppBar` emite `TabSelected`, pero en `MainPage.xaml` esta asi:

```
<layout:BottomAppBar Grid.Row="2"/>
```

Para conectar el evento deberia quedar similar a:

```
<layout:BottomAppBar Grid.Row="2" TabSelected="OnTabSelected"/>
```

Ademas se deben registrar rutas reales para `routes`, `location` y `profile`, o ajustar los nombres a las rutas existentes.

Profile con namespace inconsistente

El archivo esta en:

```
Features/Profile/Views/Profile.xaml
```

Pero usa:

```
namespace app_movil.Features.Home.Views;
```

Funciona si todo apunta a ese namespace, pero para orden del proyecto conviene cambiarlo a:

```
namespace app_movil.Features.Profile.Views;
```

Datos estaticos

La tarjeta de ruta muestra datos fijos. Para una version mas completa se recomienda que esos datos vengan de:

- Un ViewModel.
- Un servicio local.
- Una API del backend.

19. Como ejecutar o validar

Desde la carpeta del proyecto movil:

```
cd "C:\Users\ROJAS\Desktop\ADS0\Guardian Escolar\dvlp-front\dvlp-movil\app-movil"
dotnet build app-movil.csproj
```

Si se usa solucion desde una carpeta superior, verificar primero donde esta el archivo `.slnx` y ejecutar:

```
dotnet build app-movil.slnx
```

Validaciones manuales recomendadas:

1. Abrir la app en emulador o dispositivo Android.
2. Confirmar que inicia en `Welcome`.
3. Presionar `Ingresar` y verificar que abre `MainPage`.
4. Verificar que el mapa carga con internet.
5. Volver al flujo de recuperacion de contrasena.
6. Probar `ForgotPassword`, `VerifyCode` y `NewPassword`.
7. Revisar que los textos se vean correctamente.
8. Revisar que no haya pantallas cortadas en resoluciones pequenas.
9. Revisar si la barra inferior responde al tocar rutas, ubicacion y perfil.
10. Validar que los permisos de internet esten activos en Android.

20. Repaso rapido de funcionamiento

Esta seccion resume para que sirve cada parte importante de la app y que codigo se debe reconocer al leer el proyecto.

20.1. Archivos principales

Archivo	Para que sirve
<code>app-movil.csproj</code>	Define que el proyecto es MAUI, para que plataforma compila y que dependencias usa.
<code>MauiProgram.cs</code>	Configura el arranque de la app, librerias, fuentes, UraniumUI, SkiaSharp y logs.
<code>App.xaml</code>	Carga estilos y colores globales.
<code>App.xaml.cs</code>	Crea la ventana principal y carga <code>AppShell</code> .
<code>AppShell.xaml</code>	Define las pantallas principales de navegacion.
<code>AppShell.xaml.cs</code>	Registra rutas para navegar entre pantallas.

20.2. Codigo importante de arranque

Este código hace que la app inicie usando MAUI y cargue sus herramientas visuales:

```
builder
    .UseMauiApp<App>()
    .UseUraniumUI()
    .UseUraniumUIMaterial()
    .UseSkiaSharp()
    .ConfigureFonts(fonts =>
    {
        fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
        fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
        fonts.AddMaterialSymbolsFonts();
    });
```

Este código crea la ventana principal:

```
return new Window(new AppShell());
```

20.3. Código importante de navegación

Las pantallas principales se declaran en `AppShell.xaml`:

```
<ShellContent
    Title="Welcome"
    ContentTemplate="{DataTemplate auth:Welcome}"
    Route="Welcome" />

<ShellContent
    Title="MainPage"
    ContentTemplate="{DataTemplate home:MainPage}"
    Route="MainPage" />
```

Las pantallas secundarias se registran en `AppShell.xaml.cs`:

```
Routing.RegisterRoute(nameof(ForgotPassword), typeof(ForgotPassword));
Routing.RegisterRoute(nameof(VerifyCode), typeof(VerifyCode));
Routing.RegisterRoute(nameof(NewPassword), typeof(NewPassword));
Routing.RegisterRoute(nameof(Profile), typeof(Profile));
```

Para navegar se usa:

```
await Shell.Current.GoToAsync(nameof(ForgotPassword));
await Shell.Current.GoToAsync("//MainPage");
```

20.4. Codigo importante del mapa

El mapa se carga en `MainPage.xaml.cs` con Mapsui:

```
var map = new Mapsui.Map();
map.Widgets.Clear();
map.Layers.Add(OpenStreetMap.CreateTileLayer());
RouteMap.Map = map;
```

La vista se centra aproximadamente sobre Colombia:

```
var (minX, minY) = SphericalMercator.FromLonLat(-81.85, -4.23);
var (maxX, maxY) = SphericalMercator.FromLonLat(-66.85, 13.51);
var bbox = new MRect(minX, minY, maxX, maxY);
map.Navigator.ZoomToBox(bbox);
```

Para que el mapa cargue, Android necesita permisos de internet:

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
```

20.5. Codigo importante de traducciones

Los textos se llaman desde XAML usando una clave:

```
Text="{services:Translation Key=Login}"
```

La clave `Login` se busca en los archivos:

```
Resources/Strings/AppStrings.es.resx
Resources/Strings/AppStrings.en.resx
Resources/Strings/AppStrings.fr.resx
Resources/Strings/AppStrings.pt.resx
```

Esto permite cambiar textos sin modificar cada pantalla manualmente.

20.6. Para que sirve cada carpeta

Carpeta	Para que sirve
<code>Features/Auth</code>	Contiene las pantallas de login y recuperacion de contrasena.
<code>Features/Home</code>	Contiene la pantalla principal con el mapa.

Carpeta	Para que sirve
Features/Profile	Contiene la pantalla base del perfil.
Components/Buttons	Guarda botones reutilizables como <code>PrimaryButton</code> y <code>NotificationButton</code> .
Components/Inputs	Guarda entradas reutilizables como <code>InputField</code> , <code>CodeInput</code> y <code>SearchInput</code> .
Components/Layout	Guarda componentes de estructura como <code>BottomTabBar</code> .
Components/Cards	Guarda tarjetas visuales como <code>RouteInfoCard</code> .
Core/Services	Guarda servicios internos como <code>LocalizationService</code> .
Core/Extensions	Guarda extensiones para usar traducciones en XAML.
Resources	Guarda estilos, colores, fuentes, imagenes, iconos, splash y textos.
Platforms	Guarda configuraciones propias de Android, iOS, Windows y MacCatalyst.

20.7. Funciones principales de la app

Funcion	Donde esta	Para que sirve
Inicio de sesion	<code>Welcome</code>	Permite entrar visualmente a la app.
Recuperar contraseña	<code>ForgotPassword</code>	Permite escribir correo para solicitar codigo.
Verificar codigo	<code>VerifyCode</code>	Permite ingresar codigo de recuperacion.
Nueva contraseña	<code>NewPassword</code>	Permite escribir y confirmar nueva contraseña.
Mapa	<code>MainPage</code>	Muestra una vista geografica con OpenStreetMap.
Buscar ruta	<code>SearchInput</code>	Prepara una entrada para buscar rutas.
Notificaciones	<code>NotificationButton</code>	Muestra acceso visual a alertas o avisos.
Informacion de ruta	<code>RouteInfoCard</code>	Muestra ruta, conductor, placa, horario, paradas y destino.
Navegacion inferior	<code>BottomTabBar</code>	Permite cambiar entre rutas, ubicacion y perfil.
Perfil	<code>Profile</code>	Pantalla base para informacion del usuario.

21. Checklist de repaso

Para estudiar el proyecto, conviene poder responder:

- Que es `.NET MAUI` y por que se usa en la app movil.
- Donde se declaran las dependencias del proyecto.
- Que hace `Mauiprogram.cs`.
- Que hace `AppShell`.
- Cuales son las pantallas del flujo de autenticacion.
- Como se navega de `Welcome` a `MainPage`.

- Como se navega de recuperacion de contrasena a codigo y nueva contrasena.
- Donde se carga el mapa.
- Para que sirve `Mapsui.Maui`.
- Para que sirve `UseSkiaSharp`.
- Donde estan los permisos de internet de Android.
- Que componentes son reutilizables.
- Para que sirve `PrimaryButton`.
- Para que sirve `InputField`.
- Para que sirve `CodeInput`.
- Para que sirve `SearchInput`.
- Para que sirve `RouteInfoCard`.
- Para que sirve `BottomAppBar`.
- Como funcionan las traducciones con `.resx`.
- Que cosas ya son funcionales y que cosas siguen siendo visuales o pendientes.

22. Resumen final

La app movil de Guardian Escolar ya tiene una base funcional importante: estructura MAUI, navegacion con Shell, flujo visual de autenticacion, recuperacion de contrasena, pantalla principal con mapa, componentes reutilizables, estilos globales y traducciones.

Las funciones nuevas mas importantes son la pantalla principal con **Mapsui/OpenStreetMap**, el buscador de rutas, el boton de notificaciones, la tarjeta de informacion de ruta, la barra inferior y la pantalla base de perfil.

El estado actual es bueno como prototipo movil navegable, pero para una version mas completa se recomienda conectar la app con el backend, corregir detalles de navegacion, limpiar namespaces, reemplazar datos estaticos por datos reales y revisar la codificacion de textos.